Conversation Summary: AI-Focused Paper Subject and Research Plan

This document summarizes the conversation between the user and Manus AI, which led to the development of a complete research plan for an AI-focused paper.

1. Initial Request and Context Analysis

User Request:

> *suggest an AI focused paper subject for the attached* `<attachment filename="pasted_content.txt" local_path="/home/ubuntu/upload/pasted_content.txt" />`

The attached file was identified as a Call for Papers (CFP) for a workshop on **Load Testing and Benchmarking of Software Systems (ICPE 2026)**.

2. Suggested AI-Focused Paper Subjects

The following subjects were proposed, categorized by their focus:

Category 1: AI-Driven Performance Engineering

Paper Subject	Description
LLM-GenLoad: Automated Generation of Realistic Performance Test Scripts using Large Language Models	Investigates using LLMs to analyze system documentation or API specs to automatically generate complex load testing scripts.
Adaptive Stress Testing of Microservices via Deep Reinforcement Learning	Proposes a framework where a Reinforcement Learning agent learns to identify the specific combination of request types and volumes that most efficiently "breaks" a microservice architecture.
Predictive Anomaly Detection in High-Volume Load Test Results using Transformer-based Architectures	Focuses on using time-series transformers to distinguish between expected performance degradation under load and actual system anomalies.

Category 2: Performance Engineering for AI Systems

Paper Subject	Description
Benchmarking the Scalability of Retrieval-Augmented Generation (RAG) Systems under Concurrent User Load	Evaluates how the latency and accuracy of RAG pipelines degrade as the number of concurrent users increases.
Performance Characterization of LLM Inference Gateways: A Comparative Study of Throughput and Latency	A benchmarking study comparing different inference serving engines under various production-like workload traces.
Energy-Aware Benchmarking of Large Language Model Deployments on Heterogeneous Hardware	Investigates the trade-offs between performance and energy consumption when deploying LLMs across different hardware tiers.

3. Expansion of "LLM-GenLoad" Subject

The user selected **LLM-GenLoad** for further expansion.

Abstract

Modern software systems, particularly those built on microservices and cloud-native architectures, require rigorous load testing to ensure performance resilience. However, the manual creation of performance test scripts remains a significant bottleneck, often resulting in scripts that fail to capture the complexity of real-world user behavior or become obsolete as APIs evolve. This paper introduces **LLM-GenLoad**, a novel framework that leverages Large Language Models (LLMs) to automate the generation of realistic, high-fidelity performance test scripts. By processing heterogeneous inputs —including OpenAPI specifications, system documentation, and historical traffic logs —LLM-GenLoad synthesizes executable scripts for popular testing tools like JMeter and Locust. Our approach employs a “Chain-of-Performance” prompting strategy to ensure that generated scripts account for critical performance engineering concerns such as session persistence, data parameterization, and think-time distributions. Preliminary evaluations suggest that LLM-GenLoad can reduce script development time by up to 70% while improving the coverage of edge-case user scenarios compared to manual authoring.

Proposed Methodology

The LLM-GenLoad framework is structured into four primary modules: **Context Extraction**, **Behavioral Modeling**, **Script Synthesis**, and **Validation & Refinement**.

Module	Input Sources	Primary Technique	Output
Context Extraction	OpenAPI (Swagger), Markdown Docs	RAG-based Semantic Parsing	System Interaction Graph
Behavioral Modeling	Traffic Logs (ELK/Splunk), User Stories	Few-Shot Prompting	Abstract User Workload Model
Script Synthesis	Workload Model, Tool Templates	Chain-of-Performance Generation	Executable Scripts (Python/XML)
Validation	Generated Scripts, Test Environment	Iterative Feedback Loop	Refined, Error-Free Test Suite

4. Feasibility Assessment for PC Simulation

User Question:

> *is this a research that I can do virtually by running simulation on a PC ?*

Assessment: Yes, the research is highly feasible to conduct virtually on a standard PC. The focus is on **script generation logic** and validation against a representative microservice, not on running massive, real-world load tests.

Recommended Software Stack:

Component	Recommended Tool	Purpose
Containerization	Docker Desktop	To run the target microservices easily.
Load Testing Tool	**Locust**	Python-based, easy for LLMs to generate and for debugging.
LLM Interface	LangChain or LiteLLM	To easily switch between different AI models.
Target SUT	Google Microservices Demo	Provides a complex, multi-service environment for testing.

5. Complete Research Plan

User Question:

> *compose the complete research plan*

Research Questions (RQs):

- **RQ1 (Correctness):** To what extent can LLMs generate executable load testing scripts from raw API specifications?
- **RQ2 (Realism):** How accurately do LLM-generated scripts reflect complex user behavior patterns?
- **RQ3 (Maintainability):** Can LLMs effectively “self-heal” performance scripts when provided with error logs or updated API documentation?

Experimental Design

- **System Under Test (SUT):** Google Microservices Demo (Online Boutique).
- **Independent Variables:** LLM Models (GPT-4o vs. Llama-3-70B), Prompting Strategy (Zero-shot vs. CoP), and Input Context.
- **Evaluation Metrics:** Compilation Rate, Assertion Success, Behavioral Fidelity (KL-Divergence), and Human Effort (Time-to-Fix).

Implementation Roadmap (8–12 Weeks)

1. **Phase 1: Environment Setup:** Deploy SUT, configure local LLM, create “Gold Standard” script.
2. **Phase 2: Framework Development:** Implement Context Extractor and Chain-of-Performance (CoP) prompting engine.
3. **Phase 3: Data Collection and Execution:** Run generation cycles, execute scripts, and capture performance metrics.
4. **Phase 4: Analysis and Writing:** Perform statistical analysis and draft the final paper.

End of Conversation Summary